

Audio Playback Experiment (MicroPython)

Audio Playback Experiment (MicroPython)

1. Audio Playback Overview
2. Core Concepts
 - 2.1 I2S Audio Bus
 - 2.2 ES8311 Codec (DAC Mode)
 - 2.3 WAV File Format
 - 2.4 ES8311 Registers and Pop Suppression
3. Project Architecture and Flow
4. Hardware Connection
 - 5.1 File Structure
 - 5.2 Macro Configuration
 - 5.3 Register Operations
 - 5.4 init() — Hardware Initialization
 - 5.5 Key Register Quick Reference
 - 5.6 DAC Volume Control
 - 5.7 WS2812 Rainbow LED Effect
 - 5.8 WAV Player
 - 5.9 Main Program Button Logic
 - 5.10 LED Drive Timing
6. Operating Procedure
 - 6.1 File Upload
 - 6.2 Preparing WAV Files
 - 6.3 Start and Run
 - 6.4 Normal Operation Demo
 - 6.5 Edge Cases
7. FAQ
8. Parameter Tuning Guide
 - 8.1 Changing the Volume
 - 8.2 Changing the Sample Rate

1. Audio Playback Overview

WAV audio playback is one of the most fundamental and important functions in embedded audio systems. Mastering core technologies such as the I2S audio bus, the ES8311 codec driver, and WAV file parsing is a prerequisite for developing products such as smart speakers, voice assistants, and alarm systems. In IoT devices, audio playback is widely used in scenarios such as **voice prompts, music playback, device alarms, and interactive voice feedback for human-machine interaction.**

This project implements a WAV audio player based on the BOOT button. It drives the ES8311 codec (DAC mode) through the I2S bus, reads mono WAV files from the Flash file system, duplicates them into stereo in software, and outputs them through the I2S TX channel, driving the speaker through the NS4150 amplifier. MCLK is generated by PWM (GPIO15, 4.096MHz). The entire code is just one `.py` file of about 200 lines.

Core capabilities:

Function	Description
File storage	MicroPython Flash file system, stores <code>.wav</code> files
Audio decoding	ES8311 DAC, 16-bit / 16kHz / stereo output
Playback control	BOOT button (GPIO0): short press to play/pause, long press to stop
LED feedback	WS2812 RGB LED: rainbow hue animation while playing, off when not playing
Amplifier driver	NS4150 Class D amplifier, enabled on GPIO7
Pop suppression	DAC hardware mute + mute flushing + silence buffer before and after playback
MCLK generation	PWM (LEDC) outputs 4.096MHz to GPIO15

2. Core Concepts

2.1 I2S Audio Bus

I2S (Inter-IC Sound) is a standard digital audio transmission protocol that includes the following signal lines:

Signal	Full Name	Function
MCLK	Master Clock	Master clock (codec working clock) = sample rate × 256
BCLK	Bit Clock	Bit clock, one pulse per data bit
WS / LRCLK	Word Select	Frame clock, low = left channel, high = right channel
DOUT	Data Out	Sends data (ESP32 → DAC)

This project only uses DOUT (playback) and does not need DIN (recording).

	MCLK (GPIO15, PWM 4.096MHz)	/ Master clock
ESP32	→	ES8311
	BCLK (GPIO16, 512kHz)	/ Bit clock
ESP32	→	ES8311
	WS (GPIO18, 16kHz)	/ Frame clock
ESP32	→	ES8311
	DOUT (GPIO8)	/ Playback data
ESP32	→	ES8311 SDIN (DAC)
	I2C (GPIO41/42)	/ Control interface
ESP32	↔	ES8311

2.2 ES8311 Codec (DAC Mode)

The ES8311 is a low-power audio codec from Everest Semiconductor. It is configured through I2C registers (address 0x18), and audio data is transmitted over I2S:

Feature	Parameter
I2C address	0x18 (7-bit)
Operating mode	DAC (playback only)
MCLK	4.096 MHz (generated by PWM, $256 \times F_s$)
Sample rate	16000 Hz
Bit depth	16-bit
Channels	Stereo slots (mono duplicated to L+R)

2.3 WAV File Format

WAV is Microsoft's waveform audio file format. Under PCM encoding it consists of a **44-byte header + raw PCM data**:

Offset	Size	Field	Value
0x00	4	"RIFF"	RIFF magic number
0x04	4	file_len	File size - 8
0x08	4	"WAVE"	WAVE identifier
0x0C	4	"fmt "	Format chunk identifier
0x10	4	16	PCM format chunk size
0x14	2	1	Audio format (1=PCM)
0x16	2	channels	Number of channels (1=mono)
0x18	4	sample_rate	Sample rate (16000 Hz)
0x1C	4	byte_rate	Byte rate (sample rate $\times$ channels $\times$ bit depth / 8)
0x20	2	block_align	Frame alignment (channels $\times$ bit depth / 8)
0x22	2	bit_depth	Bit depth (16)
0x24	4	"data"	Data chunk identifier
0x28	4	data_len	Number of PCM data bytes

In this project the WAV file is 16kHz / 16-bit / mono, and is duplicated into stereo output in software during playback.

2.4 ES8311 Registers and Pop Suppression

The player only uses the ES8311 DAC path and does not enable the ADC. Initialization uses the same complete register sequence as the Repeater Experiment project, and mute buffering is performed both before and after playback to suppress pops.

Pop suppression flow:

```
Init phase:
Step 1: DAC mute (REG31 bits[6:5]=11)
Step 2: Register baseline config → clock source selection → divider → I2S format
→ 16-bit
Step 3: Enable DAC + power up analog/digital
Step 4: Mute flush ~1 second of zero PCM → PA on (DAC stays muted)

Play start:
① dac_vol(VOL)           → set target volume (REG32)
② dac_mute(False)        → unmute hardware (REG31)
③ i2s.write(1024B silence) → leading buffer

Play end:
① i2s.write(1024B silence) → trailing buffer
② dac_mute(True)          → mute again
```

When pausing/resuming, only the REG31 mute bit is controlled; the I2S data stream is left untouched:

```
# Pause – mute immediately
dac_mute(True)

# Resume – unmute first
dac_mute(False)
```

3. Project Architecture and Flow

```
main()
├─ init()           ← Hardware initialization
│  ├─ PA pin pulled low (amplifier off)
│  ├─ PWM outputs 4.096MHz MCLK (GPIO15)
│  ├─ I2C bus + device scan (confirm 0x18)
│  ├─ I2S TX started first (generates BCLK+WS)
│  ├─ es8311_open()   – register baseline config
│  ├─ config_sample() – clock divider (4.096MHz → 16kHz)
│  ├─ I2S format + 16-bit – REG09/REG0A config
│  ├─ es8311_start()  – enable DAC + power up
│  └─ Pop suppression – mute → flush → PA on
│
├─ RainbowLED()      ← WS2812 initialization
├─ wavPlayer()        ← Player instance
└─ btn main loop      ← button polling (~50Hz), LED driver

btn main loop:
while True:
    Detect BOOT button (GPIO0, active low)
```

```

Debounce 10ms
|
├─ Short press (<800ms) → play/pause
|   1. Idle → _thread starts play()
|   2. Playing + not paused → pause (dac_mute(True))
|   3. Playing + paused → resume (dac_mute(False))
|
└─ Long press (≥800ms) → stop
    Set stop_req=True, playback thread exits by itself

```

```

play() (independent thread):
open WAV → parse header (RIFF→fmt→data)
→ dac_vol(VOL)          set volume
→ dac_mute(False)       unmute
→ 1024B leading silence buffer  prevent pops
→ while data available:
|   └─ f.readinto(mono)  stream read 1024B
|   └─ mono→stereo      duplicate to L+R
|   └─ i2s.write(stereo) write to I2S
|   when paused: while paused: sleep
→ 1024B trailing silence buffer  prevent pops
→ dac_mute(True)          mute again
→ f.close()

```

4. Hardware Connection

Pin	Function	Connection Target
GPIO42	I2C SCL	ES8311 SCL
GPIO41	I2C SDA	ES8311 SDA
GPIO15	MCLK	ES8311 MCLK (PWM output 4.096MHz)
GPIO16	I2S BCLK	ES8311 BCLK
GPIO18	I2S WS	ES8311 LRCK (frame clock)
GPIO8	I2S DOUT	ES8311 SDIN (DAC data input)
GPIO7	PA_EN	NS4150 amplifier enable (active high)
GPIO48	WS2812 Data	RGB LED data input
GPIO0	BOOT button	Active low, internal pull-up

Core devices:

Device	Model	Communication Interface	Address/Pin
MCU	ESP32-S3	—	—
Audio codec	ES8311	I2C + I2S	I2C address 0x18
Amplifier	NS4150	GPIO control	GPIO7 (enable high)
RGB LED	WS2812	Single-wire	GPIO48

5.1 File Structure

Complete source code: [5.Source Code Collection](#) → [MicroPython](#) → [5.Audio](#) → [1.Audio_Playing](#) → [main.py](#)

This project uses a single-file design, with all code concentrated in `audio_player.py`:

```
Flash /
├─ audio_player.py    ← player main program
├─ *.wav              ← WAV audio files (placed in root directory)
```

5.2 Macro Configuration

All hardware pins and audio parameters are centrally managed at the top of the file:

```
# ===== Pins =====
SCL=42; SDA=41; PA_PIN=7; ADDR=0x18
SCK=16; WS=18; DOUT=8; MCLK_PIN=15
LED_PIN=48; BTN=0

# ===== Audio =====
RATE=16000; VOL=70
```

Macro	Value	Description
<code>RATE</code>	16000	Sample rate / Hz
<code>VOL</code>	70	Playback volume percentage (upper limit 70%)
<code>MCLK_PIN</code>	15	MCLK output pin, PWM generates 4.096MHz
<code>ADDR</code>	0x18	ES8311 7-bit I2C device address

5.3 Register Operations

The ES8311 registers are read and written directly via I2C. Two one-line functions wrap MicroPython's `writeto_mem` / `readfrom_mem`:

```
def _w(r,v): _i2c.writeto_mem(ADDR, r, bytes([v])) # write register
def _r(r):   return _i2c.readfrom_mem(ADDR, r, 1)[0] # read register
```

5.4 init() — Hardware Initialization

`init()` is the entry point of the audio chain and executes strictly in order: MCLK clock established → I2C bus → I2S started first to generate BCLK+WS → ES8311 register configuration → pop suppression.

Step 1: Amplifier off + MCLK + I2C + I2S

The following code completes four basic hardware initializations — PA pulled low to prevent pops, PWM outputs 4.096MHz MCLK, I2C bus ready, I2S TX started first to generate BCLK+WS:

```
# 1. Amplifier off
_pa = Pin(PA_PIN, Pin.OUT, value=0); time.sleep_ms(100)

# 2. MCLK — PWM 4.096MHz on GPIO15
_mclk = PWM(Pin(MCLK_PIN), freq=4096000, duty=512)
time.sleep_ms(5)

# 3. I2C bus
_i2c = I2C(0, scl=Pin(SCL), sda=Pin(SDA), freq=100000); time.sleep_ms(30)

# 4. I2S master starts first, generates BCLK+WS
_i2s = I2S(0, sck=Pin(SCK), ws=Pin(WS), sd=Pin(DOUT),
          mode=I2S.TX, bits=16, format=I2S.STEREO, rate=RATE, ibuf=11520)
_i2s.write(bytearray(1024))
```

Why start I2S before ES8311? As a slave device, the ES8311's internal state machine needs the BCLK and WS clock signals to latch the register configuration.

Step 2: es8311_open() — Register Baseline Configuration

The following code writes the necessary register groups such as system power mode, GPIO noise immunity, microphone gain, and CSM reset:

```
_w(0x0D, 0xFA); _w(0x44, 0x08); _w(0x44, 0x08)
_w(0x01, 0x30); _w(0x02, 0x00); _w(0x03, 0x10); _w(0x04, 0x10); _w(0x05, 0x00)
_w(0x16, 4); _w(0x0B, 0x00); _w(0x0C, 0x00); _w(0x10, 0x1F); _w(0x11, 0x7F)
_w(0x00, 0x80)
_w(0x01, 0x3F); _w(0x06, _r(0x06) & ~0x20)
_w(0x13, 0x10); _w(0x1B, 0x0A); _w(0x1C, 0x6A); _w(0x44, 0x58)
```

Step 3: config_sample + I2S Format

Configure the clock dividers ($MCLK \div 1$, $BCLK \div 4$, $LRCK \div 256$) and the standard I2S 16-bit format:

```
# === config_sample (4.096M, 16k) ===
_w(0x02, (_r(0x02) & 0x07) | (0<<5) | (0<<3)); _w(0x05, 0x00)
_w(0x03, (_r(0x03) & 0x80) | (0<<6) | 0x10)
_w(0x04, (_r(0x04) & 0x80) | 0x20)
_w(0x07, (_r(0x07) & 0xC0) | 0x00); _w(0x08, 0xFF)
_w(0x06, (_r(0x06) & 0xE0) | 3)

# === I2S format + 16-bit ===
_w(0x09, _r(0x09) & 0xFC); _w(0x0A, _r(0x0A) & 0xFC)
_w(0x09, _r(0x09) | 0x0C); _w(0x0A, _r(0x0A) | 0x0C)
```

Step 4: es8311_start() — Enable DAC

Slave mode, clear tri-state gates, power up analog/digital supplies, ADC/DAC ramp rates:

```
_w(0x00, 0x80); _w(0x01, 0x3F)
v9=_r(0x09)&0xBF; vA=_r(0x0A)&0xBF; v9&=0xC0; vA&=0xC0
_w(0x09,v9); _w(0x0A,vA)
_w(0x17,0xBF); _w(0x0E,0x02); _w(0x12,0x00); _w(0x14,0x1A)
_w(0x14,_r(0x14)&~0x40); _w(0x0D,0x01); _w(0x15,0x40); _w(0x37,0x08);
_w(0x45,0x00)
```

Step 5: Pop Suppression

DAC mute → minimum volume → 1 second silence flush through the I2S pipeline → amplifier on (DAC stays muted):

```
dac_mute(True); _w(0x32, 0x00)
_i2s.write(bytearray(12800)); time.sleep_ms(50)
_pa.on(); time.sleep_ms(100)
```

5.5 Key Register Quick Reference

Register	Name	Function	Typical Value
0x00	RESET	Reset and master/slave mode	0x80 (slave mode)
0x01	CLK_MANAGER1	MCLK source selection + clock enable	0x3F (MCLK pin, all clocks on)
0x02	CLK_MANAGER2	Pre-divider + multiplier	pre=1, multi=1x
0x03	CLK_MANAGER3	ADC fs_mode + ADC OSR	OSR=0x10
0x04	CLK_MANAGER4	DAC OSR	OSR=0x20
0x06	CLK_MANAGER6	BCLK inversion + BCLK division	BCLK÷4
0x09	SDPIN	DAC I2S format + bit depth + tri-state	I2S + 16-bit
0x0A	SDPOUT	ADC I2S format + bit depth	Same as 0x09
0x0D	SYSTEM0D	System power up/down	0x01=power up
0x0E	SYSTEM0E	Analog power (VMID reference)	0x02
0x12	SYSTEM12	DAC path enable	0x00=enabled
0x31	DAC31	DAC mute bits[6:5]	0x60=muted, 0x00=normal
0x32	DAC32	DAC volume	0x00=min (-95.5dB), 0xFF=max (+32dB)

5.6 DAC Volume Control

`dac_mute()` and `dac_vol()` are the two most frequently called interfaces of the player. Muting provides hardware-level cutoff via REG31 bits[6:5], and volume is mapped through REG32 (hard-clamped to a 70% upper limit, consistent with the Arduino version of the ES8311 database):

```
def dac_mute(m):
    """DAC mute (REG31)"""
    _w(0x31, (_r(0x31) & 0x9F) | (0x60 if m else 0))

def dac_vol(p):
    """DAC volume 0-70% → REG32 (max 70%)"""
    if p > 70: p = 70
    _w(0x32, int(p * 255 / 100))
```

REG32 Value	Percentage	Corresponding dB
0x00	0%	-95.5 dB (mute)
0xB2 (178)	70%	≈ -6.5 dB
0xFF	100%	+32 dB (maximum)

REG32 and dB have a **linear mapping** (not exponential), approximately 0.5 dB per step.

5.7 WS2812 Rainbow LED Effect

Based on MicroPython's built-in `neopixel` library, the hue space 0–65535 is divided into 6 sectors, advancing about 1.4° per step. Note the GRB wire order uses `(g, r, b)` rather than `(r, g, b)`:

```
class RainbowLED:
    def __init__(self): self.np=Neopixel(Pin(LED_PIN),1); self.hue=0
    def step(self):
        s=(self.hue>>13)&7; x=(self.hue>>5)&0xFF
        if s==0: r,g,b=255,x,0 # Red→Yellow
        elif s==1: r,g,b=255-x,255,0 # Yellow→Green
        elif s==2: r,g,b=0,255,x # Green→Cyan
        elif s==3: r,g,b=0,255-x,255 # Cyan→Blue
        elif s==4: r,g,b=x,0,255 # Blue→Magenta
        else: r,g,b=255,0,255-x # Magenta→Red
        self.np[0]=(g,r,b); self.np.write()
        self.hue=(self.hue+400)&0xFFFF
    def off(self): self.np[0]=(0,0,0); self.np.write()
```

5.8 WAV Player

The `WavPlayer` class encapsulates the complete playback lifecycle — opening the file, parsing the header, preparing the volume, streaming read in the main loop → stereo duplication → I2S write, and leading/trailing silence buffers. `play()` runs in an independent thread and communicates with the button main loop through the global flags `playing/paused/stop_req`.

Playback main loop / play():

The following code shows the complete WAV playback flow — open file and verify RIFF header → iterate chunks to locate fmt and data → set volume and unmute → 1024B leading silence → read 1024B mono chunk by chunk → duplicate each sample to L+R → write to I2S:

```
def play(self, path):
    if self.playing: return False
    self.playing=True; self.paused=False; self.stop_req=False

    try: f=open(path,"rb")
    except: print(f"[WAV] open failed: {path}"); self.playing=False; return False

    # Parse header
    if f.read(4)!=b'RIFF' or f.read(8)[4:]!=b'WAVE':
        print("[WAV] bad header"); f.close(); self.playing=False; return False

    rate=ch=bits=16000; nb=0
    while True:
        tag=f.read(4); cs=int.from_bytes(f.read(4),'little')
        if tag==b'fmt ':
            d=f.read(cs); _,ch,rate,_,_,bits=struct.unpack('<HHIIHH',d[:16])
        elif tag==b'data': nb=cs; break
        else: f.read(cs)

    dac_vol(VOL); dac_mute(False); time.sleep_ms(10)
    _i2s.write(bytearray(1024)) # leading silence

    mono=bytearray(1024); stereo=bytearray(2048); rem=nb
    while rem>0 and not self.stop_req:
        while self.paused and not self.stop_req: time.sleep_ms(50)
        if self.stop_req: break
        n=f.readinto(memoryview(mono)[:min(1024,rem)])
        if n<=0: break; rem-=n
        ns=n//2
        for i in range(ns):
            lo=mono[i*2]; hi=mono[i*2+1]
            stereo[i*4]=lo; stereo[i*4+1]=hi
            stereo[i*4+2]=lo; stereo[i*4+3]=hi
        _i2s.write(stereo[:ns*4])

    _i2s.write(bytearray(1024)) # trailing silence
    dac_mute(True); f.close()
    self.playing=False; self.paused=False; self.stop_req=False
```

Mono→Stereo Conversion Diagram:

The WAV file is mono 16-bit PCM, 2 bytes per sample; the ES8311 is configured for stereo slots, so each frame must write both L and R 16-bit values. The conversion logic duplicates each 16-bit sample into the right channel:

```
Mono file: {M0, M1, M2, M3, ...}
           ↓ duplicate to L+R
Stereo output: {M0,M0, M1,M1, M2,M2, M3,M3, ...}
```

Pause/Resume / pause_resume():

When paused, only the REG31 mute bit is set; the I2S is not stopped and the file is not closed — resuming simply unmutes for a seamless continuation:

```
def pause_resume(self):
    if not self.playing: return False
    self.paused=not self.paused
    dac_mute(self.paused)
    return not self.paused
```

Stop / stop():

Sets the `stop_req` flag and releases the pause; the playback thread's main loop detects it and exits on its own, cleaning up resources:

```
def stop(self):
    if not self.playing: return
    self.stop_req=True; self.paused=False
```

5.9 Main Program Button Logic

The button uses polling (~50Hz) and detects the press and release edges. The press edge records the time, and the release edge determines the duration — <800ms is a short press (play/pause toggle), ≥800ms is a long press (stop). The long press has an instant response mechanism and does not wait for release:

```
prev=1; done=False; down=0; tick=0
while True:
    raw=btn.value(); now=time.ticks_ms()
    if raw==0 and prev:      # press edge
        time.sleep_ms(10)
        if btn.value()==0: prev=0; done=False; down=now
    elif raw==1 and not prev: # release edge
        prev=1
        if not done:
            held=time.ticks_diff(now,down)
            if held<LONG_MS: # short press
                if player.is_busy(): player.pause_resume()
                else: _thread.start_new_thread(player.play,(path,))

            # immediate long press response
            if not prev and not done and time.ticks_diff(now,down)>=LONG_MS:
                done=True; player.stop()
```

Button Behavior Table:

Action	Current State	Result
Short press (<800ms)	Idle	Start playback (LED rainbow)
Short press (<800ms)	Playing	Pause (DAC muted, LED off)
Short press (<800ms)	Paused	Resume playback (DAC unmuted, LED rainbow)
Long press (≥800ms)	Playing/Paused	Stop playback (close file, DAC muted)

5.10 LED Drive Timing

The rainbow effect is updated in the main loop at approximately 60ms intervals — the main loop runs at 50Hz (every 20ms), calling `led.step()` once every 3 ticks:

```
if player.is_playing():
    tick+=1
    if tick>=3: led.step(); tick=0
else: tick=0; led.off()
```

6. Operating Procedure

6.1 File Upload

Upload `audio_player.py` and the `.wav` files to the ESP32-S3 Flash root directory:

```
Flash /
├─ audio_player.py      ← player script
├─ song1.wav            ← WAV audio files
└─ song2.wav
```

Use Thonny IDE's file manager to upload. For detailed steps, see the "Environment Setup" chapter:

MicroPython → 1. Before Development → 1.Environment Setup → Environment Setup

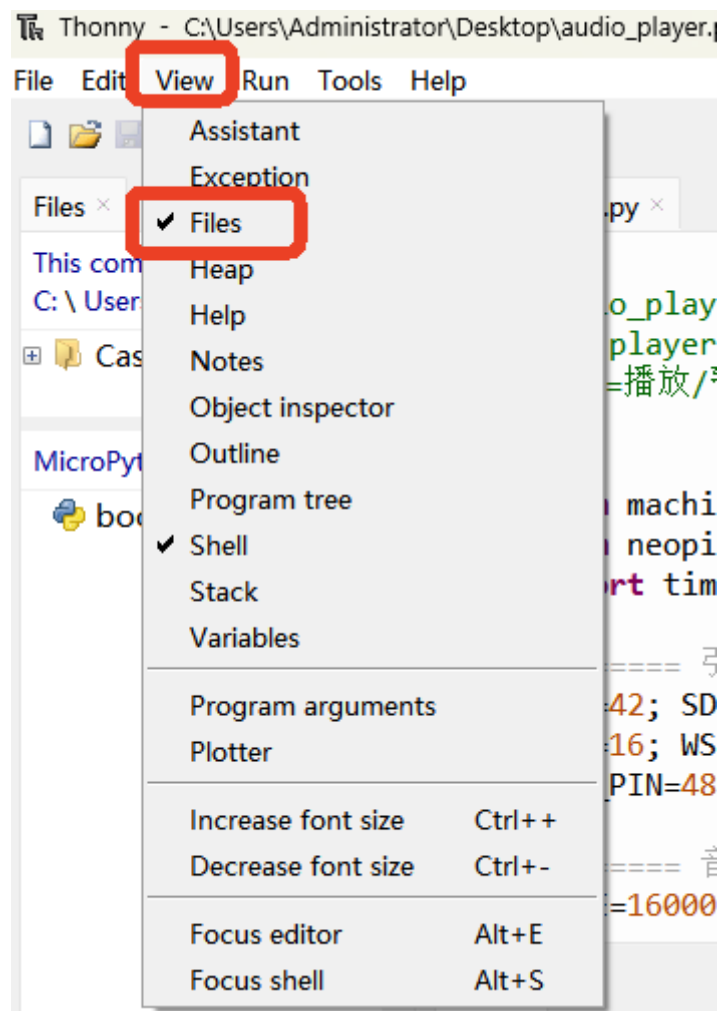
6.2 Preparing WAV Files

WAV file requirements:

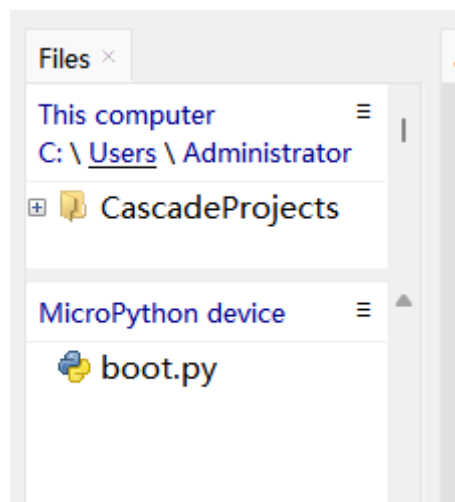
Parameter	Requirement
Encoding	PCM
Sample rate	16000 Hz
Bit depth	16-bit
Channels	Mono (recommended) or stereo

Place the files into Flash:

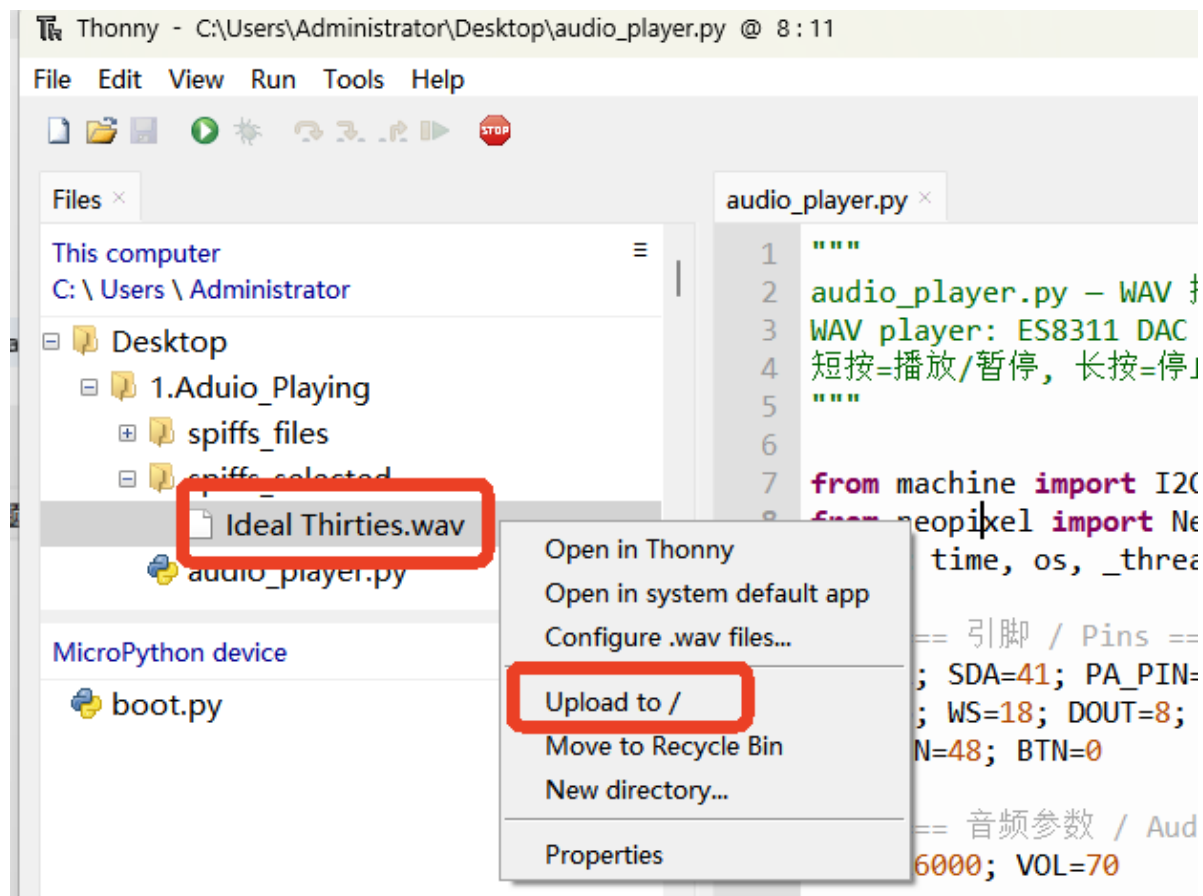
1. Open the file manager: "View" → "File"



2. You will see the file management area on the left; locate the path of your wav file



3. Upload the wav file



6.3 Start and Run

Enter the following in the REPL:

```
import audio_player
audio_player.main()
```

The program automatically scans the `.wav` files in the root directory and selects the first one as the default playback file.

6.4 Normal Operation Demo

Step 1: Power-on reset, observe the serial output

```
===== Audio Player =====
I2C: ['0x18']
ES8311 OK - MCLK=PWM, PA=ON, DAC=MUTED
WAV: Ideal Thirties.wav
Short=Play/Pause | Long=Stop
```

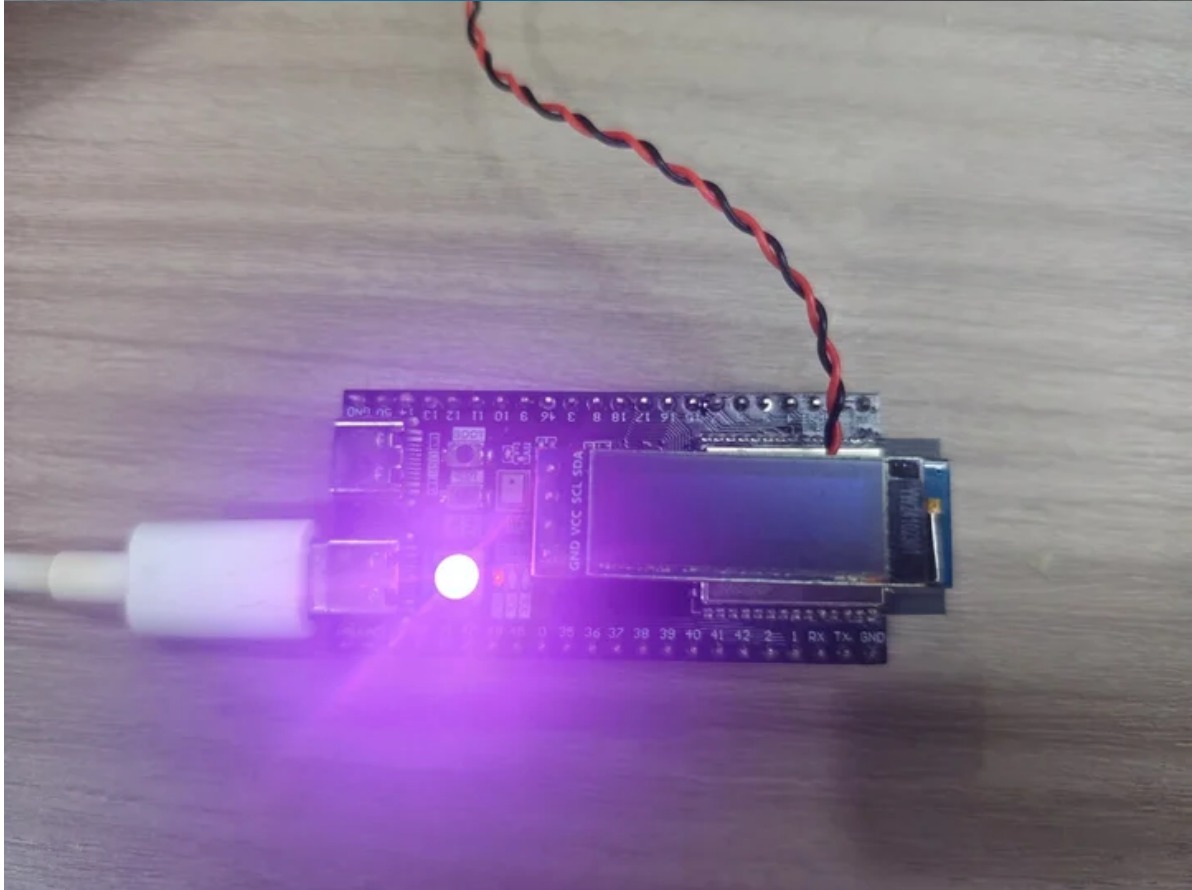
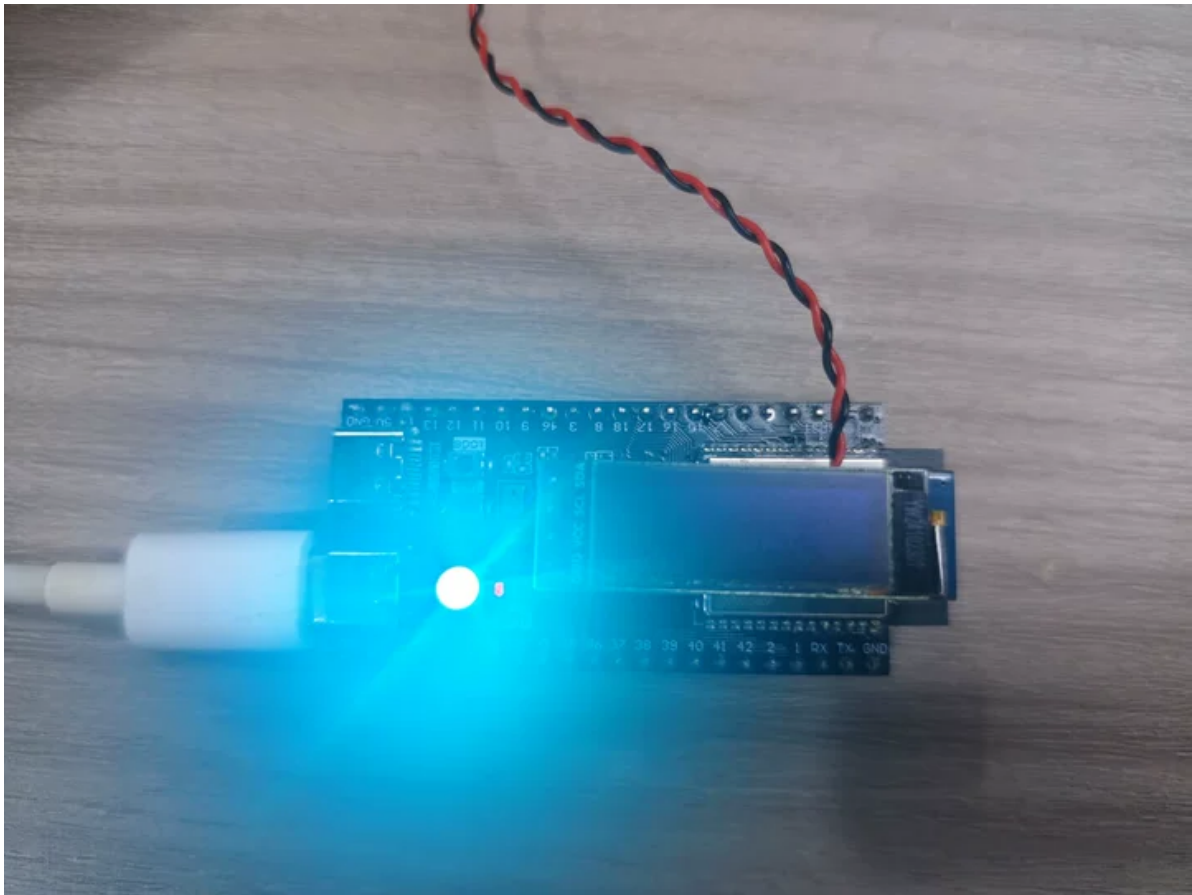
```
I2C: ['0x18']
ES8311 OK - MCLK=PWM, PA=ON, DAC=MUTED
WAV: Ideal Thirties.wav
Short=Play/Pause | Long=Stop
```

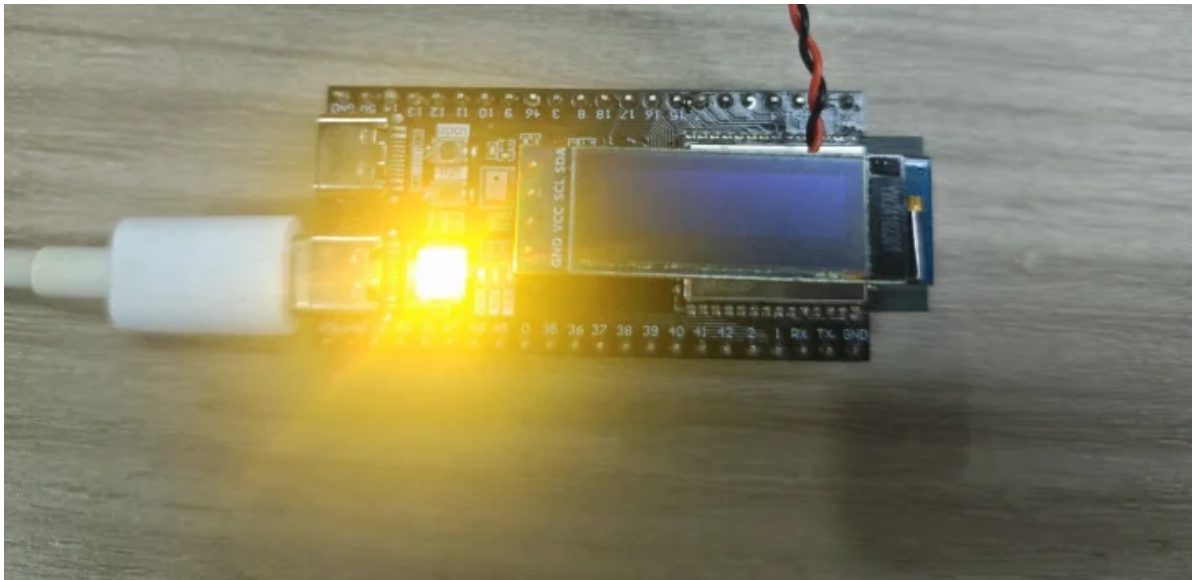
Step 2: Short press the BOOT button to start playback

```
BTN: Play
[WAV] /Ideal Thirties.wav: 16000Hz 1ch 16bit 729600B
```

```
[WAV] /Ideal Thirties.wav: 16000Hz 1ch 16bit 2560000B
```

The LED shows a flowing rainbow effect and the speaker outputs sound.





Step 3: Short press the BOOT button while playing

- First short press → pause, DAC muted, LED off
- Press again → resume, DAC unmuted, LED rainbow

Step 4: Long press the BOOT button (≥0.8s)

Playback stops immediately, the file is closed, the DAC is muted, and the LED turns off.

6.5 Edge Cases

Scenario	Expected Behavior
Short press with no WAV file	Prints "open failed", no exception
File is not WAV format	Prints "bad header", playback terminates
Long press to stop	Playback thread exits, file closed correctly
Playback reaches end of file	Automatically stops, mute restored
Abnormal crash	DAC restored to mute, file closed, flags reset

7. FAQ

Symptom	Cause	Solution
I2C scan does not show 0x18	ES8311 not powered / I2C wiring error	Check the SCL(42)/SDA(41) wire order
No sound during playback	PA not on / I2S clock not running / volume set to 0	Measure whether GPIO7 is high, increase VOL
Pops during playback	Pop suppression sequence missing	Confirm the mute → flush → PA on flow in init()
Abnormal playback speed	Sample rate mismatch	Ensure the WAV file is 16000Hz, consistent with RATE
Abnormal WS2812 color	RGB/GRB wire order error	Already written as (g,r,b) in the code
Button not responding	GPIO0 occupied / debounce too short	Check whether other functions occupy GPIO0
Playback stutters	Insufficient I2S buffer / slow Flash reads	Increase ibuf or reduce CHUNK

8. Parameter Tuning Guide

8.1 Changing the Volume

```
# audio_player.py top
VOL = 70      # Playback volume 0-70 (upper limit 70%)
```

8.2 Changing the Sample Rate

```
# audio_player.py top
RATE = 48000  # Hz (the WAV file must use the corresponding sample rate)
```